

Local Rules, Global Efficiency: Emergent Coordination on Large-Scale Road Networks

Zhenhui Jessie Li

Hong Kong Polytechnic University

Hong Kong

jessielzh@gmail.com

Abstract

Urban transportation systems face a fundamental efficiency paradox: individually optimal routing decisions can create system-wide congestion that increases travel time for all participants. Traffic engineers address this through rigid infrastructure solutions—for example, roundabouts that enforce circular flow, or turn restrictions that prevent conflicting maneuvers. However, these manually designed interventions lack flexibility: optimized for peak demand, they unnecessarily constrain traffic during off-peak hours. Autonomous vehicles (AVs) offer a transformative opportunity for dynamic coordination that adapts to traffic conditions. However, achieving such adaptive coordination at city scale with thousands of decentralized vehicles remains an open challenge.

We propose **Guided Local Coordination (GLC)**, a decentralized framework combining global shortest-path guidance with priority-based local conflict resolution. GLC precomputes optimal distance labels for the entire network, then uses priority inheritance and backtracking to resolve real-time collisions without replanning. We demonstrate that GLC scales to city-scale networks with 4K nodes and 10K agents, achieving high performance. As a key qualitative result, GLC produces emergent roundabout-like behavior: under high density, left-turn traffic spontaneously forms circulating flows while straight-through traffic proceeds unimpeded—adaptive coordination arising purely from local rules without explicit programming.

CCS Concepts

• Computing methodologies → Multi-agent planning.

Keywords

autonomous vehicles, multi-agent coordination, road networks, traffic routing, emergent behavior

ACM Reference Format:

Zhenhui Jessie Li. 2026. Local Rules, Global Efficiency: Emergent Coordination on Large-Scale Road Networks. In *Proceedings of the 32nd ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '26)*, August 09–13, 2026, Jeju Island, Republic of Korea. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3770855.3818071>



This work is licensed under a Creative Commons Attribution 4.0 International License. *KDD '26, Jeju Island, Republic of Korea*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2259-2/2026/08
<https://doi.org/10.1145/3770855.3818071>

1 Introduction

Urban transportation systems face a fundamental efficiency paradox. During peak hours, busy intersections become bottlenecks where individually optimal routing decisions—such as taking the shortest path or making direct left turns—create system-wide congestion that paradoxically increases everyone’s travel time. Traffic engineers have long recognized this paradox and developed counterintuitive solutions: roundabouts that smooth high-demand intersection flow by replacing high-conflict maneuvers with structured circulation, and peak-hour restrictions on left turns. While these interventions dramatically improve overall throughput when traffic density is high, they can waste time during off-peak hours by unnecessarily constraining individual routing efficiency [12, 53].

The advent of connected and autonomous vehicles (CAVs) presents an unprecedented opportunity to move beyond these rigid, manually crafted infrastructure solutions. In a city populated by CAVs capable of real-time coordination, routing and interaction decisions are no longer made by independent human drivers, but by computational systems that can, in principle, coordinate to improve global efficiency. Vehicles could dynamically adapt their routes based on current traffic density—taking detours during peak hours and using direct routes during off-peak hours—without requiring fixed roundabouts or permanent turn restrictions. This flexibility could significantly reduce travel times, save fuel, and improve city-scale traffic efficiency. However, achieving such coordination at scale—with tens of thousands of vehicles making decentralized decisions on city-scale road networks—presents a fundamental computational challenge.

This paper addresses the problem of coordinating large populations of goal-directed agents on road networks, where each agent has a specific origin and destination and must navigate through a directed graph representing roads and intersections. While this problem shares similarities with multi-agent pathfinding (MAPF), traffic assignment, and swarm coordination, the road network setting presents unique computational challenges that distinguish our work from these established research directions.

The MAPF literature has developed sophisticated algorithms for coordinating agents in free space or grid environments with obstacles [9, 25, 34, 43]. However, conflicts on road networks are fundamentally different from conflicts in free space. Free-space MAPF typically offers uniform local connectivity (often 4 or 8 neighbors), allowing agents to sidestep each other with small detours. In contrast, road networks are *highly constrained directed graphs*: an agent at an intersection may have only 1–3 legal outgoing edges, and shortest paths often follow narrow corridors with few alternatives. As a result, many interactions become unavoidable bottlenecks that

require coordination along constrained routes rather than simple spatial rerouting.

In transportation research, the Traffic Assignment Problem (TAP) seeks to find User Equilibrium (UE) solutions where no driver can improve their travel time by unilaterally changing routes [36, 44]. While TAP methods are designed for large-scale road networks, they typically require iterative optimization procedures that become computationally expensive with tens of thousands of agents, limiting their applicability to real-time coordination scenarios. Dynamic variants can be substantially more expensive, often requiring hours to days at metropolitan scale [37]. Moreover, TAP focuses on equilibrium routing rather than local coordination at the granularity of individual vehicle movements.

The swarm intelligence literature has demonstrated that simple local rules can produce complex emergent collective behaviors in biological and robotic systems [6, 40]. However, these studies typically focus on abstract coordination tasks (e.g., flocking, aggregation, foraging) rather than goal-directed navigation on structured road networks. To our knowledge, no prior work has investigated whether swarm-like emergent patterns—such as spontaneous formation of roundabout-like flows at intersections—can arise from decentralized coordination algorithms applied to road network navigation.

We propose **Guided Local Coordination (GLC)**, a decentralized planning framework that addresses the unique challenges of road network coordination through a novel combination of global guidance and local conflict resolution. GLC operates in two stages: first, it precomputes shortest-path distance labels for the entire road network using efficient graph algorithms, providing each agent with global directional guidance toward its goal. Second, during execution, GLC employs a priority-based local conflict resolver that dynamically resolves collisions through priority inheritance and backtracking—allowing higher-priority agents to push lower-priority agents out of the way while maintaining progress toward goals. This two-stage design is specifically tailored to road networks: the precomputation phase exploits the static graph structure to provide optimal guidance efficiently, while the local resolution phase handles the dynamic, constrained conflicts that arise from the directed, bottleneck-prone nature of road networks. Critically, GLC achieves this coordination without requiring global replanning, inter-agent communication, or learning, making it computationally tractable for large-scale real-time deployment.

Our key contributions are:

- **Large-scale road network coordination.** We demonstrate that GLC scales to realistic urban networks, achieving efficient coordination on a Manhattan-style grid network with 4K nodes and 10K agents.
- **Emergent roundabout-like behavior.** As a key qualitative result, we show that in a four-way intersection scenario, GLC produces an emergent roundabout-like pattern for left-turn traffic while preserving straight-through flow—effectively “inventing” roundabout-like behavior as a decentralized consequence of local conflict resolution, without explicit programming or learning. This demonstrates that sophisticated coordination patterns observed in swarm systems can emerge in goal-directed road network navigation.
- **Theoretical foundations.** We provide formal analysis characterizing GLC’s computational cost and establishing per-timestep feasibility and recovery guarantees for the PIBT-style local resolver, which explain how GLC resolves conflicts without global replanning.

Our results suggest that carefully designed decentralized coordination algorithms can overcome the computational barriers that have limited prior work, enabling scalable real-time coordination for autonomous vehicles in urban environments. By bridging insights from MAPF, transportation research, and swarm intelligence, GLC opens new directions for designing coordination systems that are both computationally efficient and capable of producing emergent collective behaviors.

2 Related Work

Our work bridges three research areas: multi-agent pathfinding (MAPF), transportation coordination, and swarm intelligence. We highlight the unique challenges of decentralized coordination on large-scale road networks.

2.1 Multi-Agent Pathfinding

The MAPF problem seeks collision-free paths for multiple agents in shared environments [46]. Optimal centralized methods such as Conflict-Based Search (CBS) [43] and its variants [3, 7, 15], as well as bounded-suboptimal approaches like Priority-Based Search [27] and Large Neighborhood Search [22], provide strong guarantees but require centralized planning and become intractable for thousands of agents. Learning-based methods [23, 42] face generalization issues across network topologies and lack interpretability for safety-critical deployment.

Decentralized approaches make local decisions without centralized coordination. WHCA* [45] uses time windows for local planning, while token passing [50] coordinates through token exchange. Priority Inheritance with Backtracking (PIBT) [34] introduced a fully decentralized framework where agents make local decisions based on dynamic priorities, achieving impressive scalability on warehouse environments with thousands of agents. Guided PIBT [33] enhanced this by using A*-based guidance and predicted congestion costs, so agents prefer moves that remain close to a goal-directed path while avoiding locally crowded regions. LaCAM [30] improves upon PIBT by lazily adding constraints only when conflicts occur, demonstrating scalability to thousands of agents on warehouse maps. Recent extensions address continuous spaces [20], dynamic environments [32], and lifelong settings [28, 31]. RHCR [24] combines global planning with local collision resolution.

However, existing MAPF methods are designed for *free-space or grid environments* with uniform connectivity (4–8 neighbors per cell) where agents can route around obstacles. In contrast, road networks are *highly constrained directed graphs* with only 1–3 legal outgoing edges due to one-way streets and turn restrictions—agents heading to nearby destinations are often forced onto the same narrow corridors with no alternative routes. While recent work reports results on maps with 38,589 traversable cells [9, 33], the high degree of freedom in these environments makes coordination easier than on road networks with even 4,000 nodes where movement is restricted to specific road sequences. To our knowledge, our work

is the first to systematically study MAPF-style coordination on realistic urban road network topologies.

2.2 Transportation Coordination

The Traffic Assignment Problem (TAP) distributes traffic across networks to achieve equilibrium states using methods like Frank-Wolfe [16, 44] that iteratively adjust routes until reaching User Equilibrium [51]. Extensions include dynamic TAP [17, 37] and stochastic variants [8, 11]. However, TAP requires *iterative optimization taking minutes to hours to converge* [2], making it unsuitable for real-time coordination where vehicles must make decisions in milliseconds.

Collective driving and platooning [21, 38] and intersection management systems [1, 14, 26] address different problems, focusing on highway scenarios or isolated intersections rather than network-wide urban navigation with diverse origin-destination pairs.

2.3 Swarm Intelligence and Emergent Behavior

Swarm intelligence studies how simple local rules produce complex collective behaviors [6]—flocking in birds [39], ant foraging [13], and fish schooling [10] demonstrate emergent coordination without centralized control. Swarm robotics applies these principles to collective transport [18], pattern formation [41], and exploration [4], with recent work on self-organized construction [52] and collective decision-making [49]. Research on emergent traffic patterns shows lane formation in bidirectional flow [19, 47] and stop-and-go waves [48].

However, swarm research addresses *abstract coordination tasks* (flocking, aggregation, foraging) rather than goal-directed navigation with specific destinations on structured road networks. While our work also demonstrates emergent roundabout-like behavior, we do so in the context of constrained, goal-directed navigation with hard collision avoidance requirements.

3 Problem and Preliminaries

3.1 Problem Definition

We consider decentralized multi-agent routing under *two simulation settings*. In both settings, the problem is defined at the coordination layer: time is discrete, agents make one routing/coordination decision per timestep, and we do not model acceleration, braking, continuous vehicle motion, or heterogeneous vehicle dynamics.

Grid (synthetic) setting. The environment is a lane-level grid map: each traversable cell represents a small road segment, and directed lanes are encoded by restricting which neighboring cells are reachable. Capacity is therefore implicit in the grid resolution: at most one agent may occupy a cell at a time (cell capacity 1), and wider roads are modeled by multiple parallel lane cells.

Graph (real road-network) setting. The environment is a directed road graph $G = (V, E)$ where V are intersections (road nodes) and each directed edge $(u, v) \in E$ is an admissible movement. To focus on network-level coordination rather than low-level car-following behavior, we do not simulate within-edge continuous motion: traversing an edge is represented as a discrete transition in the main experiments.

Input. A grid map or directed graph, and a set of start/goal pairs $\{(s_i, g_i)\}_{i=1}^n$.

Output. A set of trajectories $\{v_i(t)\}_{t \geq 0}$ such that $v_i(0) = s_i$ and each agent reaches its goal at time T_i with $v_i(T_i) = g_i$. Equivalently, a decentralized policy that maps each agent’s local state to an action at each timestep.

Objective and evaluation (total travel time). For each agent i , let T_i denote its arrival timestep (the first t such that $v_i(t) = g_i$). We evaluate methods using **total travel time** $\sum_i T_i$.

3.2 Dynamics and Constraints

Discrete-time dynamics. Time is discretized as $t = 0, 1, 2, \dots$. At each timestep, each active agent either waits or moves one step (to an adjacent cell in the grid setting, or along one directed edge in the graph setting).

Grid (synthetic road network) feasibility. We enforce single-occupancy cells (capacity 1). There is no explicit edge-capacity model; reservations can be viewed as a set of occupied cells.

Graph (real city road network) feasibility. We enforce node capacity constraints that allow multiple agents to occupy an intersection node u up to an integer capacity $\text{cap}(u)$ computed from the number of incoming lanes (with a bounded range, e.g., 2–12). We also enforce a directed edge capacity $\text{cap}(u \rightarrow v)$ limiting how many agents may traverse (u, v) in the same timestep.

3.3 Modeling Assumptions and Scope

Methodological scope and abstraction. This work deliberately adopts discrete-time dynamics and simplified vehicle models to isolate the fundamental coordination problem from vehicle physics and control implementation details. This abstraction serves three purposes: (1) it enables formal complexity analysis and theoretical guarantees (see Sec. 4), (2) it provides a reproducible benchmark for comparing coordination algorithms at scale, and (3) it allows us to demonstrate emergent coordination patterns in controlled settings without confounding factors from continuous dynamics or heterogeneous vehicle behaviors.

Modeling philosophy. Our discrete-time model with unit edge traversal represents a deliberate simplification that prioritizes coordination logic over vehicle dynamics. This approach is standard in multi-agent pathfinding research [43, 46] and enables analysis of coordination patterns at scales (10K agents) that would be computationally prohibitive with full physics simulation. The model can be interpreted as a coarse-grained representation where each timestep corresponds to a fixed travel duration (e.g., 2–3 seconds at typical urban speeds), and edge weights can be adjusted to reflect speed limits or road lengths.

Limitations and extensions. Real-world deployment would require extensions to handle acceleration and braking, heterogeneous vehicle characteristics, richer queueing/spillback effects, and mixed autonomy settings with non-compliant human drivers. We treat these as important future directions and discuss practical pathways in the conclusion.

Algorithm 1 GLC high-level step (per timestep)

Require: Positions $v_i(t)$, goals g_i , directed graph G , feasibility predicate FEASIBLE

Ensure: Next positions $v_i(t+1)$

- 1: **Guidance:** for each active agent i , compute/refresh the goal-distance label $D_i(\cdot) = \text{dist}(\cdot, g_i)$ as needed
- 2: rank candidates $\{v_i(t)\} \cup N^+(v_i(t))$ by increasing D_i
- 3: **Conflict resolution:** process agents in priority order and apply the local conflict resolver to select a feasible joint move
- 4: satisfying FEASIBLE
- 5: **Replanning trigger:** if agent i deviates from its preferred D_i -decreasing successor due to backtracking,
- 6: refresh its guidance before the next decision

4 Method: Guided Local Coordination (GLC)

We propose Guided Local Coordination (GLC), a decentralized planner that combines (i) *global routing guidance* based on directed shortest paths and (ii) a *local conflict resolver* that enforces feasibility under contention. Algorithm 1 summarizes the per-timestep framework.

4.1 Global Routing Guidance

Shortest-path goal-distance labels. For each agent i with goal g_i , we compute the directed shortest-path distance to the goal, $D_i(v) = \text{dist}(v, g_i)$. Absent interference, repeatedly choosing an outgoing edge that decreases D_i follows a shortest path.

Aggregate progress. Define the aggregate goal-distance $\Phi(t) = \sum_{i \in \mathcal{A}(t)} D_i(v_i(t))$, where $\mathcal{A}(t)$ is the set of active (not-yet-arrived) agents at time t . If, at time t , every active agent moves along a shortest-path edge (decreasing its label by 1), then $\Phi(t+1) = \Phi(t) - |\mathcal{A}(t)|$. When contention prevents all agents from simultaneously decreasing D_i , GLC still uses D_i as a globally consistent ranking signal and delegates feasibility to the local resolver.

Relation to Guided PIBT. GLC intentionally combines simple existing ingredients rather than introducing a new local resolver: its contribution is showing that shortest-path graph guidance plus lightweight PIBT-style local coordination is sufficient for large directed road networks. Relative to Guided PIBT [33], the key difference is the form of global guidance. GLC uses shortest-path labels on the road graph, whereas Guided PIBT uses A^* -based guidance with predicted congestion costs to steer agents toward less crowded routes. Thus, Guided PIBT places more computation in global planning, while GLC keeps global guidance simple and resolves interactions through local coordination; this distinction is reflected in the 100-agent Manhattan experiment, where Guided PIBT requires 49.82s while GLC requires only 0.69s.

4.2 Local Conflict Resolver

We instantiate GLC's local conflict resolver using Priority Inheritance with Backtracking (PIBT) [34]. At a high level, PIBT processes agents in priority order. When a high-priority agent requests a node occupied by a lower-priority agent, the blocking agent temporarily *inherits* the higher priority and is recursively assigned a move first.

If the recursion cannot find a feasible assignment (due to reservations, swaps, or road constraints), PIBT backtracks and tries the next alternative.

Priorities. In the original PIBT formulation, priorities are adaptive: an agent's priority increases as it continues to wait (preventing starvation) and is reset when it reaches its goal [34]. In our experiments, we instantiate the priority order using distance-to-go, assigning higher priority to agents with smaller remaining distance, which aligns with our shortest-path guidance and helps clear agents that are close to their destinations. This is an efficiency-oriented design choice rather than a claim of universal optimality: other priority rules, such as waiting-time-based priority, can also be effective and preserve the qualitative emergent roundabout-like patterns observed in our intersection experiments.

PIBT in our road-network setting. Operationally, PIBT constructs a feasible joint move at each timestep by processing agents in priority order. For each agent i at node u , it considers the candidate set $\{u\} \cup N^+(u)$ (wait or traverse one outgoing edge) and selects the most preferred feasible successor. If the preferred successor w is currently occupied by an undecided, lower-priority agent j , PIBT invokes *priority inheritance* by recursively deciding j first. If the recursion fails (because no feasible assignment can be found given the current reservations), PIBT backtracks and tries the next candidate.

Departures from the original PIBT formulation. Our implementation follows the core inheritance/backtracking mechanism of Okumura et al. [34], but makes two instantiations to match road networks: (i) we operate on a *directed* graph and restrict candidates to outgoing edges $N^+(u)$ (encoding one-way streets and turn rules), and (ii) we enforce a road-specific feasibility predicate FEASIBLE(u, w) that captures both standard MAPF constraints and additional node/edge capacities and directions derived from lane information. In addition, rather than using adaptive waiting-based priorities, we order agents by distance-to-go in our directed road graphs.

Properties (informal). At each timestep, the local resolver returns a joint move that satisfies FEASIBLE by construction (via reservations and backtracking); if a feasible joint move exists within the current one-step candidate sets, the backtracking search can recover it. Formal statements and proof sketches are provided below.

4.3 Theoretical Properties

The results below formalize what the PIBT-style local resolver guarantees at the *single-timestep* level. They do not, by themselves, imply global completeness over an entire run, since feasibility and priorities change over time.

THEOREM 4.1 (PER-TIMESTEP FEASIBILITY AND RECOVERY). *Fix a timestep t and a priority ordering over the active agents. Let each active agent i at state $v_i(t)$ have a finite candidate set $C_i := \{v_i(t)\} \cup N^+(v_i(t))$ (wait or traverse one outgoing edge). Assume the feasibility predicate FEASIBLE depends only on the proposed next states (and, in the Manhattan setting, the corresponding traversed edges) at timestep t . If there exists a joint assignment $(x_i)_{i \in \mathcal{A}(t)}$ with $x_i \in C_i$ for all i*

such that *FEASIBLE* holds, then the recursive inheritance/backtracking search used by our PIBT instantiation can return a feasible joint move.

PROOF SKETCH. Under a fixed priority order, the resolver performs a depth-first search over the finite Cartesian product $\prod_i C_i$. When a candidate choice for an agent conflicts with already-reserved resources, it is rejected; when it conflicts with an undecided agent’s current occupancy, the recursion decides the blocking agent first (priority inheritance). If a downstream choice later proves infeasible, the algorithm backtracks and tries the next candidate. Because the search space is finite and the algorithm explores alternatives via backtracking, it can recover any feasible joint assignment consistent with the fixed priority order and candidate sets. $\square$

4.4 Computational Cost and Scalability

Let $N = |\mathcal{A}(t)|$ be the number of active agents and let $|V|, |E|$ denote the road-graph size. Computing a distance label $D_i(\cdot)$ requires a shortest-path routine (Breadth-First Search (BFS) on unweighted graphs, Dijkstra otherwise): $O(|V| + |E|)$ for BFS and $O(|E| + |V| \log |V|)$ for Dijkstra. The local conflict resolver considers only immediate outgoing neighbors and uses recursion/backtracking to decide moves, so per-timestep overhead is dominated by local feasibility checks rather than repeated global replanning.

Why GLC scales. GLC separates expensive graph-wide computation (shortest-path distances) from local contention handling. Distance labels can be computed once per agent per (re)planning event, whereas the local resolver operates on constant-size neighborhoods. In contrast, guided methods that invoke congestion-aware global replanning during conflict resolution may pay a graph-wide cost many times per timestep, which becomes prohibitive on road networks with thousands of nodes.

5 Experiments

We evaluate GLC on (i) synthetic road-like settings that highlight qualitative emergent coordination patterns and (ii) a city-scale Manhattan road network with real taxi OD demand to quantify performance under realistic topology and capacity constraints. The source code is publicly available online ¹.

5.1 Experimental Setup and Metrics

Compute resources. All experiments were conducted on a MacBook Air equipped with an Apple M5 system-on-chip and 24 GB unified memory. Wall-clock runtimes reported in the tables correspond to this hardware configuration.

Primary metric (total travel time). We report **total travel time** $\sum_i T_i$, where T_i is the timestep when agent i reaches its destination. Equivalently, each timestep an agent remains active (not yet arrived), its travel-time counter increases by 1. Lower total travel time indicates faster overall completion and captures both waiting time (blocked by others) and movement time.

¹<https://github.com/jessielzh/GLC-road-coordination/>

IDEAL baseline and overhead. We compute each agent’s BFS shortest-path length d_i^* on the directed road graph *ignoring other agents*. The **IDEAL** baseline $\sum_i d_i^*$ is a theoretical lower bound (zero interference). We report overhead as $\frac{\sum_i T_i}{\sum_i d_i^*} - 1$.

5.2 Baseline Methods

We compare against baselines motivated by three perspectives. **IDEAL** provides the lower bound needed to compute overhead.

Transportation-inspired policies capture manually designed, infrastructure-like interventions (e.g., roundabouts).

- **SP (shortest path + waiting).** A common routing baseline: each agent computes a shortest path once at spawn and follows it step-by-step, waiting whenever the next state is infeasible.
- **Circular (transportation coordination).** A manually designed roundabout policy: any agent entering the intersection circulates counter-clockwise (CCW) until it reaches its desired exit. This mimics real-world roundabouts and turn-management practice, but can impose unnecessary detours (especially for through and right-turn traffic). Due to its specialized design, this policy applies only to the four-way intersection scenario and is evaluated only in that setting.
- **TAP (traffic assignment).** A classical traffic-assignment baseline (e.g., UE-style assignment) that captures network-level coordination at an aggregate level [36, 44]. TAP fits network settings (link flows, BPR-style costs, UE assignment) much better than grid-cell settings (discrete cells, one-agent-per-cell dynamics, and no natural notion of link flow or cost). We evaluate TAP only in the Manhattan road-network setting.

MAPF baselines represent scalable local conflict-resolution methods from the multi-agent pathfinding literature.

- **PIBT.** A scalable state-of-the-art local conflict-resolution method based on priority inheritance and backtracking [34] (see details in Sec. 4.2).
- **G-PIBT.** An advanced PIBT variant that uses A*-based guidance to improve local decision making [33]. This is conceptually related to our use of global guidance, but was proposed for free-space MAPF settings.

5.3 Emergent Pattern Discovery

5.3.1 Scenario: Four-way 2-in-1-out intersection. We simulate a four-way intersection on a 64×64 grid with right-hand driving (Fig. 1). Each arm has two lanes: one inbound lane toward the intersection and one outbound lane away from it. Within the intersection, agents may turn to reach any outbound lane. The intersection size is 11×11 grid cells. We spawn 24 agents at the far ends of inbound lanes and assign destinations on outbound lanes according to turn type (30% left, 35% straight, 35% right). This scenario is highly conflict-prone because left turns cross opposing traffic and the intersection is a shared resource.

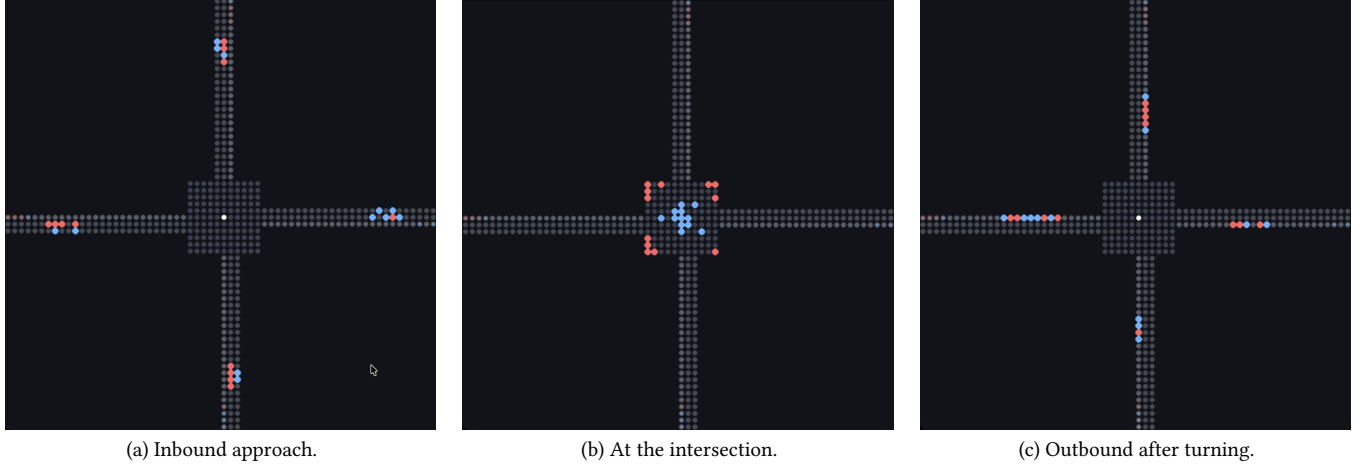


Figure 1: Snapshots of an emergent roundabout arising from purely local conflict resolution. Red agents are left-turn vehicles; blue agents travel straight through. Instead of cutting directly across opposing traffic, the left-turn agents spontaneously form a one-way ring inside the intersection — circulating until they reach their outbound lane — a roundabout that no agent was programmed to create.

Table 1: Four-way intersection results averaged over three seeds. We report mean total travel time $\sum_i T_i$ with relative overhead over the mean IDEAL baseline in parentheses.

Method	$\sum_i T_i$ (overhead) ↓
IDEAL	1,423
SP	1,885 (+32%)
Circular	1,724 (+21%)
PIBT	1,457 (+2%)
G-PIBT	1,474 (+4%)
GLC (ours)	1,475 (+4%)

5.3.2 Quantitative Results. As shown in Table 1, SP performs worst among the listed methods (+32%) because uncoordinated shortest-path execution creates repeated conflicts between through movements and left turns, leading to excessive waiting. Circular incurs substantial overhead (+21%), reflecting the cost of a manually designed roundabout that sacrifices straight-through traffic by forcing all vehicles to circulate. In this local intersection setting, PIBT, G-PIBT, and GLC perform similarly (about +2–4% overhead): the A*-guided variants behave close to PIBT because the dominant interactions are local and the shortest-path guidance provides limited additional benefit; their advantages are more pronounced in larger, global-network settings.

Figure 2 highlights why a fixed roundabout rule can be inefficient: when left-turn demand is low (i.e., as the left-turn ratio approaches 0), forcing every vehicle to circulate adds avoidable distance and creates entry/exit bottlenecks, even though many movements could otherwise pass directly.

5.3.3 Roundabout-Like Pattern Emerges. For visual clarity, this illustrative run uses 24 agents with a slightly modified demand mix consisting only of left-turn and straight-through vehicles (50% each), so the two vehicle colors directly distinguish the two movement types. In the four-way intersection, left-turn traffic forms a stable

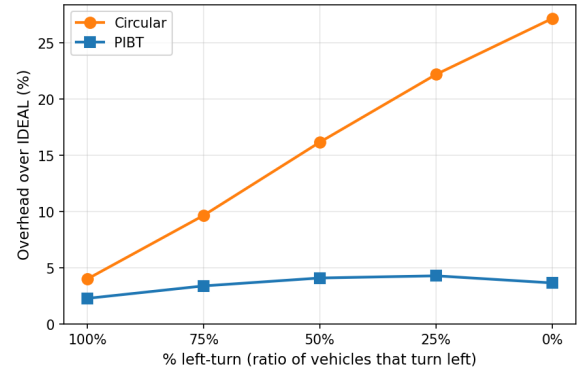


Figure 2: Overhead over IDEAL versus the left-turn ratio in the four-way intersection. As left-turn demand decreases, the hard-coded Circular policy incurs increasing overhead because it forces unnecessary circulation even when through movements are conflict-free. In contrast, PIBT remains low-overhead because left-turn vehicles tend to adopt roundabout-like motion while through vehicles proceed directly.

roundabout-like pattern while through traffic proceeds straight (Fig. 1), suggesting that roundabout-like behavior can emerge as a decentralized consequence of local conflict resolution.

Direct left turns create the highest-conflict interactions: they block through traffic and require gaps in opposing flow. Under PIBT-style “push” resolution, blocked agents propagate pressure backward along lanes; in dense settings, this pressure is relieved when agents accept short detours that preserve forward progress for everyone. The intersection interior provides a small loop that supports monotone motion: once agents begin to traverse the perimeter

in a consistent direction, conflicts become mostly merge-like (rather than crossing), enabling steady throughput.

5.4 Small Road Network Experiment



Figure 3: Scenario: 3×3 directed grid. Sampled agent locations (blue: origins; red: destinations).

5.4.1 Scenario 2: 3×3 directed grid. We simulate an 80×80 grid containing nine 3×3 “tiny intersections” arranged in a 3×3 pattern, connected by two-lane roads (Fig. 3). Each lane is single-direction under right-hand driving: horizontally, the top lane goes West and the bottom lane goes East; vertically, the left lane goes South and the right lane goes North. Road segments contain 10 cells between intersections. We vary the number of agents with random spawn and destination positions on road lanes (distinct from spawn), producing a mix of through movements and left turns. This scenario emphasizes narrow corridors, strict lane discipline, and the difficulty of executing left turns in very small intersections.

Table 2: 3×3 directed grid results. We report total travel time $\sum_i T_i$ with relative overhead over IDEAL in parentheses.

Agents	IDEAL	SP	PIBT	G-PIBT	GLC
8	248	257 (+3%)	248 (+0%)	248 (+0%)	248 (+0%)
16	511	530 (+4%)	514 (+1%)	515 (+1%)	514 (+1%)
24	795	825 (+4%)	798 (+0%)	802 (+1%)	800 (+1%)
32	1,055	1,096 (+4%)	1,061 (+1%)	1,072 (+2%)	1,063 (+1%)
40	1,297	1,352 (+4%)	1,309 (+1%)	1,328 (+2%)	1,311 (+1%)

5.4.2 Quantitative Results. Table 2 shows that PIBT, G-PIBT, and GLC generally achieve near-optimal performance on the 3×3 directed grid, with overheads mostly between 0–3%. SP remains competitive at lower densities (3–5% overhead), suggesting that severe deadlocks are rare in this small setting and that simple waiting often suffices to resolve local contention.

The small gap between PIBT-style methods is expected because the environment is regular and short. However, the *computational* differences between guidance schemes are not visible at this scale. In our city-scale experiments, where shortest-path computations span thousands of nodes and conflicts recur over long horizons, the extra global search performed by G-PIBT becomes prohibitive,

while GLC maintains scalability by amortizing global guidance and relying on local resolution.

5.5 City-Scale Experiments

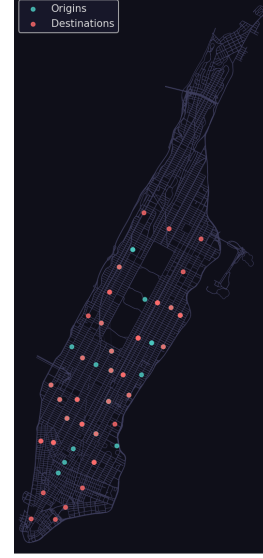


Figure 4: Scenario: Manhattan road network with 4,594 nodes and 9,856 edges. Sampled agent origins and destinations are shown for illustration only.

5.5.1 Scenario: Manhattan road network (real taxi OD). We evaluate at city scale on a directed road network extracted from OpenStreetMap (OSM) using OSMnx (drive network, simplified, projected). The resulting Manhattan network has 4,594 nodes and 9,856 directed edges, with one-way streets represented by edge direction. OD demand is derived from the NYC Taxi & Limousine Commission (TLC) Yellow Taxi Trip Record Data by filtering to trips whose pickup and drop-off are both in Manhattan taxi zones, dropping same-zone trips, and sampling up to N OD pairs for a given run (different random seeds yield different samples) [5, 29, 35].

We map each taxi zone to the road graph by snapping the zone centroid to the nearest road node, yielding OD pairs as (origin node, destination node). Unlike previous scenarios, we do not model grid cells: each timestep, an agent either traverses exactly one directed edge to an adjacent node or waits. We enforce (i) *intersection (node) capacity*, the maximum number of agents allowed at a node after a step, computed from OSM lane metadata by summing incoming lanes and setting $\text{cap} = \min\{12, \max\{2, \max(\lfloor \frac{\text{total_lanes}}{2} \rfloor, \text{street_count})\}\}$, and (ii) *edge capacity* $\text{cap}(u \rightarrow v)$, the maximum number of agents that may traverse edge $u \rightarrow v$ in one timestep (from per-direction lanes, default 1). After agents propose moves, we admit proposals subject to current node and edge reservations; blocked agents wait, and each agent’s time increments by 1. We report total travel time, overhead versus IDEAL, and wall-clock runtime.

Table 3: Manhattan road-network results. Total travel time is $\sum_i T_i$; overhead is relative to the IDEAL lower bound $\sum_i d_i^*$. $TAP^\dagger$ and $G-PIBT^\ddagger$ are included only in the 100-agent setting because they are orders of magnitude slower than the other methods, so we omit them at larger scales.

Method	$\sum_i T_i$ (overhead) ↓	Runtime (s) ↓
100 agents		
IDEAL	2,457	0.02
SP	2,607 (+6.1%)	0.26
$TAP^\dagger$	2,698 (+9.8%)	6.51
PIBT	2,522 (+2.6%)	0.76
$G-PIBT^\ddagger$	2,633 (+7.2%)	49.82
GLC	2,600 (+5.8%)	0.69
1,000 agents		
IDEAL	23,850	0.19
SP	33,250 (+39.4%)	4.55
PIBT	30,573 (+28.2%)	11.42
GLC	32,062 (+34.4%)	11.08
10,000 agents		
IDEAL	243,125	2.12
SP	2,203,068 (+806.1%)	618.78
PIBT	1,075,988 (+342.6%)	741.36
GLC	1,017,513 (+318.5%)	630.91

5.5.2 Quantitative Results. Table 3 highlights three trends. First, at small scale (100 agents), all methods remain relatively close to IDEAL in total travel time (2.6–9.8% overhead), but runtime differs substantially: $G-PIBT$ is much slower because it relies on heavier A^* -based, congestion-aware guidance. TAP is also much slower because user-equilibrium assignment repeatedly recomputes congestion-aware shortest paths, making it substantially more expensive than one-pass methods. In contrast, SP is the fastest simulated baseline (0.26s) but incurs higher travel time than PIBT and GLC, whereas PIBT achieves the lowest travel time (+2.6%) at a modest runtime cost (0.76s). GLC remains close to SP in runtime (0.69s) while improving over SP in total travel time (+5.8% vs. +6.1%).

Second, at 1,000 agents, capacity constraints increase travel time substantially for all methods. PIBT attains lower travel time than GLC (+28.2% vs. +34.4%), while GLC is slightly faster (11.08s vs. 11.42s). SP remains the fastest method (4.55s) but incurs the highest travel-time overhead among the decentralized methods (+39.4%).

Third, at 10,000 agents, congestion dominates and the gap between naive routing and coordination becomes extreme: SP degrades to 806.1% overhead, while PIBT and GLC remain substantially better. Among the scalable baselines reported, GLC achieves the best total travel time (318.5% overhead) with runtime close to SP (630.91s vs. 618.78s) and lower than PIBT (741.36s).

Figure 5 compares runtime progress and accumulated travel time on the 10K-agent Manhattan instance. SP completes latest, at roughly step 880, with 618.78s runtime and 2.20M total travel time. PIBT completes much earlier, at roughly step 370, but has the highest runtime (741.36s) and reaches 1.08M total travel time. GLC completes at roughly the same time as PIBT while achieving the best travel-time result (1.02M) and lower runtime (630.91s), showing that

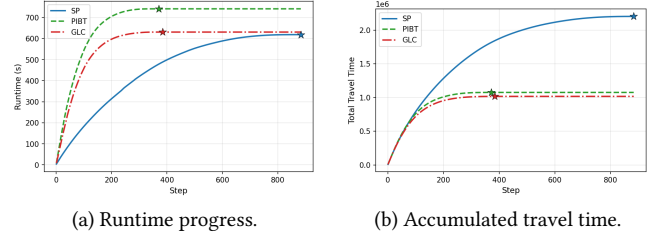


Figure 5: Manhattan road-network diagnostics for the 10K-agent setting: runtime progress and effectiveness measured by accumulated travel time. Stars mark the timestep at which all agents complete their trips. GLC achieves the best travel-time performance with runtime close to SP and lower than PIBT.

it preserves PIBT-like coordination benefits with computational cost close to SP.

6 Conclusion and Future Work

In this work, we have introduced Guided Local Coordination (GLC), a decentralized coordination framework that combines global shortest-path guidance with priority-based local conflict resolution. Our experiments across synthetic grids and a large-scale Manhattan road network have demonstrated that GLC achieves strong travel-time performance while remaining computationally tractable, even at high agent counts. Importantly, our method also yields qualitatively meaningful emergent behaviors, such as the spontaneous formation of roundabout-like behavior, without any explicit programming.

This paper is only a first step toward understanding how local coordination rules can produce globally efficient traffic behavior on road networks. It raises several important directions for future work. First, the model should be extended toward more realistic traffic settings, including continuous driving dynamics, heterogeneous vehicle behavior, road conditions, signals, and richer queueing or spillback effects. Second, mixed-autonomy settings deserve deeper study, since non-compliant or partially coordinated human drivers may change both efficiency and safety outcomes. Third, future work should examine the fairness trade-off between global efficiency and individual sacrifice: system-level improvements may require some agents to take detours or wait longer, and principled mechanisms are needed to balance these effects. Finally, stronger theoretical guarantees are needed to characterize when local rules converge, when emergent roundabout-like patterns arise, and how far decentralized coordination can be from globally optimal traffic flow.

By addressing these future research directions, we believe that the GLC framework can be further refined and extended to enable scalable, efficient, and adaptive coordination of autonomous vehicles in large-scale urban transportation networks. The insights gained from this work, which bridges MAPF, transportation research, and swarm intelligence, open up new avenues for designing coordination systems that are both computationally efficient and capable of producing emergent collective behaviors.

References

- [1] Tsz-Chiu Au, Neda Shahidi, and Peter Stone. 2015. Autonomous intersection management for semi-autonomous vehicles. In *Handbook of Transportation*. Routledge, 88–104.
- [2] Hillel Bar-Gera. 2002. Origin-based algorithm for the traffic assignment problem. *Transportation Science* 36, 4 (2002), 398–417.
- [3] Max Barer, Guni Sharon, Roni Stern, and Ariel Felner. 2014. Suboptimal variants of the conflict-based search algorithm for the multi-agent pathfinding problem. In *Proceedings of the International Symposium on Combinatorial Search*, Vol. 5. 19–27.
- [4] Levent Bayindir. 2016. A review of swarm robotics tasks. *Neurocomputing* 172 (2016), 292–321.
- [5] Geoff Boeing. 2017. OSMnx: A Python package to work with graph-theoretic OpenStreetMap street networks. *Journal of Open Source Software* 2, 12 (2017), 215.
- [6] Eric Bonabeau, Marco Dorigo, and Guy Theraulaz. 1999. *Swarm intelligence: from natural to artificial systems*. Oxford University Press.
- [7] Eli Boyarski, Ariel Felner, Roni Stern, Guni Sharon, David Tolpin, Oded Betzalel, and Solomon Eyal Shimony. 2015. ICBS: Improved conflict-based search algorithm for multi-agent pathfinding. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 740–746.
- [8] Anthony Chen and R Jayakrishnan. 2001. A self-adaptive projection and contraction algorithm for the traffic assignment problem with path-specific costs. *European Journal of Operational Research* 135, 1 (2001), 27–41.
- [9] Zhe Chen, Daniel Harabor, Jiaoyang Li, and Peter J Stuckey. 2024. Traffic flow optimisation for lifelong multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 20674–20682.
- [10] Iain D Couzin, Jens Krause, Richard James, Graeme D Ruxton, and Nigel R Franks. 2002. Collective memory and spatial sorting in animal groups. *Journal of Theoretical Biology* 218, 1 (2002), 1–11.
- [11] Gary A Davis and Nancy L Nihan. 1993. Large population approximations of a general stochastic traffic assignment model. *Operations Research* 41, 1 (1993), 169–178.
- [12] Andre J Deprator, Olivia Hitchcock, and Vikash V Gayah. 2017. Improving urban street network efficiency by prohibiting conflicting left turns at signalized intersections. *Transportation Research Record* 2622, 1 (2017), 58–69.
- [13] Marco Dorigo, Mauro Birattari, and Thomas Stutzle. 2006. Ant colony optimization. *IEEE Computational Intelligence Magazine* 1, 4 (2006), 28–39.
- [14] Kurt Dresner and Peter Stone. 2008. A multiagent approach to autonomous intersection management. *Journal of Artificial Intelligence Research* 31 (2008), 591–656.
- [15] Ariel Felner, Jiaoyang Li, Eli Boyarski, Hang Ma, Liron Cohen, TK Kumar, and Sven Koenig. 2018. Adding heuristics to conflict-based search for multi-agent path finding. In *Proceedings of the International Conference on Automated Planning and Scheduling*, Vol. 28. 83–87.
- [16] Marguerite Frank and Philip Wolfe. 1956. An algorithm for quadratic programming. *Naval Research Logistics Quarterly* 3, 1-2 (1956), 95–110.
- [17] Terry L Friesz, David Bernstein, Nihal J Mehta, Roger L Tobin, and Sima Ganjalizadeh. 1989. Dynamic network traffic assignment considered as a continuous time optimal control problem. *Operations Research* 37, 6 (1989), 893–901.
- [18] Roderich Gross and Marco Dorigo. 2009. Towards group transport by swarms of robots. *International Journal of Bio-Inspired Computation* 1, 1-2 (2009), 1–13.
- [19] Dirk Helbing. 2001. Traffic and related self-driven many-particle systems. *Reviews of Modern Physics* 73, 4 (2001), 1067.
- [20] Shao-Hung Huang, Yen-Chi Kuo, I-Chen Yen, Cheng-Wei Chen, and I-Chen Wu. 2023. Continuous multi-agent path finding via satisfiability modulo theories. *arXiv preprint arXiv:2304.06168* (2023).
- [21] Dongyao Jia, Kejie Lu, Jianping Wang, Xiang Zhang, and Xuemin Shen. 2016. A survey on platoon-based vehicular cyber-physical systems. *IEEE Communications Surveys & Tutorials* 18, 1 (2016), 263–284.
- [22] Jiaoyang Li, Graeme Gange, Daniel Harabor, Peter J Stuckey, Hang Ma, and Sven Koenig. 2021. MAPF-LNS: Large neighborhood search for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12029–12037.
- [23] Jiaoyang Li, Daniel Harabor, Peter J Stuckey, Ariel Felner, Hang Ma, and Sven Koenig. 2021. Learning to guide cost-based heuristic search. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12353–12362.
- [24] Jiaoyang Li, Wheeler Ruml, and Sven Koenig. 2021. Lifelong multi-agent path finding for online pickup and delivery tasks. In *Proceedings of the International Conference on Autonomous Agents and Multiagent Systems*. 837–845.
- [25] Jiaoyang Li, Andrew Tinka, Scott Kiesel, Joseph W Durham, TK Kumar, and Sven Koenig. 2021. EECBS: A bounded-suboptimal search for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 12353–12362.
- [26] Liang Li and Fei-Yue Wang. 2018. Signal-free intersection control: An intersection control approach for autonomous vehicles. *IEEE Transactions on Intelligent Transportation Systems* 19, 4 (2018), 1301–1310.
- [27] Hang Ma, Daniel Harabor, Peter J Stuckey, Jiaoyang Li, and Sven Koenig. 2019. Searching with consistent prioritization for multi-agent path finding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 7643–7650.
- [28] Hang Ma, Wolfgang Hoenig, TK Kumar, Nora Ayanian, and Sven Koenig. 2021. Lifelong multi-agent path finding in large-scale warehouses. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35. 11272–11281.
- [29] New York City Taxi and Limousine Commission (TLC). 2024. TLC Trip Record Data. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Yellow taxi trip records; parquet files. Accessed for Manhattan OD sampling.
- [30] Keisuke Okumura. 2023. Improving LaCAM for scalable eventually optimal multi-agent pathfinding. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 5382–5390.
- [31] Keisuke Okumura. 2023. Traffic flow optimisation for lifelong multi-agent path finding. *arXiv preprint arXiv:2308.11234* (2023).
- [32] Keisuke Okumura, François Bonnet, Xavier Défago, and Yasumasa Tamura. 2023. MAPF-IR: Multi-agent path finding with inertial constraints. *Autonomous Agents and Multi-Agent Systems* 37, 1 (2023), 1–28.
- [33] Keisuke Okumura, Xavier Défago, and Yasumasa Tamura. 2020. Guided pibt: Improving prioritized planning with backtracking for multi-agent path finding. In *IEEE International Conference on Robotics and Automation*. 9114–9120.
- [34] Keisuke Okumura, Manao Machida, Xavier Défago, and Yasumasa Tamura. 2019. Priority inheritance with backtracking for iterative multi-agent path finding. In *Proceedings of the International Joint Conference on Artificial Intelligence*. 535–542.
- [35] OpenStreetMap contributors. 2024. Planet dump retrieved from OpenStreetMap. <https://www.openstreetmap.org>. Road network for Manhattan; data © OpenStreetMap contributors, ODbL.
- [36] Michael Patriksson. 1994. The traffic assignment problem: Models and methods. VSP (1994).
- [37] Srinivas Peeta and Athanasios K Ziliaskopoulos. 2001. Foundations of dynamic traffic assignment: The past, the present and the future. *Networks and Spatial Economics* 1, 3 (2001), 233–265.
- [38] Jeroen Ploeg, Steven Shladover, Henk Nijmeijer, and Nathan Van De Wouw. 2014. Introduction to the special issue on the 2011 Grand Cooperative Driving Challenge. *IEEE Transactions on Intelligent Transportation Systems* 15, 4 (2014), 1431–1433.
- [39] Craig W Reynolds. 1987. Flocks, herds and schools: A distributed behavioral model. *ACM SIGGRAPH Computer Graphics* 21, 4 (1987), 25–34.
- [40] Michael Rubenstein, Christian Ahler, and Radhika Nagpal. 2012. Kilobot: A low cost scalable robot system for collective behaviors. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.
- [41] Michael Rubenstein, Alejandro Cornejo, and Radhika Nagpal. 2014. Programmable self-assembly in a thousand-robot swarm. *Science* 345, 6198 (2014), 795–799.
- [42] Guillaume Sartoretti, Justin Kerr, Yunfei Shi, Glenn Wagner, TK Kumar, Sven Koenig, and Howie Choset. 2019. PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning. In *2019 International Conference on Robotics and Automation (ICRA)*. IEEE, 5958–5964.
- [43] Guni Sharon, Roni Stern, Ariel Felner, and Nathan R Sturtevant. 2015. Conflict-based search for optimal multi-agent pathfinding. *Artificial Intelligence* 219 (2015), 40–66.
- [44] Yosef Sheffi. 1985. *Urban transportation networks: Equilibrium analysis with mathematical programming methods*. Prentice-Hall.
- [45] David Silver. 2005. Cooperative pathfinding. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, Vol. 1. 117–122.
- [46] Roni Stern, Nathan Sturtevant, Ariel Felner, Sven Koenig, Hang Ma, Thayne Walker, Jiaoyang Li, Dor Atzmon, Liron Cohen, TK Kumar, et al. 2019. Multi-agent pathfinding: Definitions, variants, and benchmarks. *arXiv preprint arXiv:1906.08291* (2019).
- [47] Toshiharu Sugawara and Kensuke Kurihara. 2021. Self-organized control of traffic signals through the formation of virtual lanes in multi-agent systems. *Artificial Life and Robotics* 26, 3 (2021), 343–351.
- [48] Yuki Sugiyama, Minoru Fukui, Macoto Kikuchi, Katsuya Hasebe, Akihiro Nakayama, Katsuhiro Nishinari, Shin-ichi Tadaki, and Satoshi Yukawa. 2008. Traffic jams without bottlenecks—experimental evidence for the physical mechanism of the formation of a jam. *New Journal of Physics* 10, 3 (2008), 033001.
- [49] Gabriele Valentini, Eliseo Ferrante, Heiko Hamann, and Marco Dorigo. 2016. Collective decision with 100 Kilobots: speed versus accuracy in binary discrimination problems. *Autonomous Agents and Multi-Agent Systems* 30, 3 (2016), 553–580.
- [50] Prasanna Velagapudi, Katia Sycara, and Paul Scerri. 2010. Decentralized prioritized planning in large multirobot teams. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*. 4603–4609.
- [51] John Glen Wardrop. 1952. Road paper. some theoretical aspects of road traffic research. *Proceedings of the Institution of Civil Engineers* 1, 3 (1952), 325–362.
- [52] Justin Werfel, Kirstin Petersen, and Radhika Nagpal. 2014. Designing collective behavior in a termite-inspired robot construction team. *Science* 343, 6172 (2014), 754–758.
- [53] Guancheng Yu and Vikash V Gayah. 2023. Simulation analysis of urban network performance under link disruptions and impacts of left-turn prohibitions. *Transportation Research Record* 2677, 11 (2023), 706–721.

A Additional Proof-of-Concept Experiments

A.1 More Realistic Traffic Dynamics

Ideally, traffic simulation would model richer vehicle physics and interaction rules (e.g., using a high-fidelity microscopic simulator such as SUMO). However, such simulators are significantly slower and are not necessary to validate the core coordination concept studied in this paper; accordingly, we use a simplified discrete-time simulator for our main experiments.

To assess whether our conclusions are sensitive to these simplifications, we implement a proof-of-concept (POC) experiment that extends the simulator with three lightweight realism upgrades: variable edge traversal times, queues on edges, and a simple headway constraint. We show that the same qualitative conclusions continue to hold under these more realistic edge dynamics.

1) *Variable edge traversal times.* Each edge has an integer travel time of 1–4 steps (from OSM length when present, otherwise from lane count). IDEAL is computed using a weighted shortest path (sum of edge travel times). During simulation, agents may occupy an edge for multiple steps; we decrement remaining travel-time each timestep until the agent reaches the head node.

2) *Queues on edges.* A link can have multiple agents in transit simultaneously. We define a queue capacity as

$$\text{cap}(u \rightarrow v) = \text{lanes}(u \rightarrow v) \times \text{travel_time}(u \rightarrow v),$$

so, for example, a 2-lane edge with 2-step travel time can host up to 4 agents. An agent may enter an edge only if its current occupancy is below this capacity.

3) *Safe headway / vehicle-length proxy.* As a simple headway constraint, at most one agent may enter a given edge per timestep.

Table 4: Proof-of-concept Manhattan experiment (100 agents). Total travel time is $\sum_i T_i$; overhead is relative to IDEAL.

Method	$\sum_i T_i$ (overhead) ↓	Runtime (s) ↓
IDEAL	3,351	0.05
SP	3,533 (+5.4%)	0.6
PIBT	3,445 (+2.8%)	2.7
GLC	3,484 (+4.0%)	1.5

Table 4 reports the POC results. All methods complete successfully, and both SP and GLC remain close to IDEAL. Overall, the results do not change the relative conclusion that coordination remains competitive under more realistic edge dynamics.

A.2 Mixed AV and Human Drivers

We also study a simple mixed-autonomy setting in the realistic Manhattan POC. Each agent is an AV with probability $1 - p$ and a “human” driver with probability p .

Agent models. AVs use the GLC method. They plan first and treat human positions as fixed obstacles. Humans follow shortest paths but (i) they exhibit *random yielding*—with probability `yield_prob` (default 0.15) they wait even when a move is feasible—and (ii) they do not participate in priority inheritance. Instead, humans select actions after AVs, using only the remaining node/edge capacity.

Optionally, we model gap acceptance by making a human move with probability $\max\{0, 1 - \beta \cdot \text{congestion_ratio}\}$ at the target.

Constraint-violation attempts. Whenever a human attempts to enter a node/edge that is already at capacity, the move is forced to wait and we count a *violation attempt* as a proxy for near-miss / non-compliance.

Metrics and sweep. We report total travel time, completion rate, violation attempts, and the realized counts (n_{AV}, n_H). We sweep $p \in \{0, 0.2, 0.4, 0.5, 0.6, 0.8, 1.0\}$.

Table 5: Mixed-autonomy proof-of-concept results on the realistic Manhattan setting (100 agents; `yield_prob` = 0.15).

p (human)	$\sum_i T_i$ ↓	Completion	Violations	n_{AV}/n_H
0.0	3,484	100%	0	100/0
0.2	3,608	100%	21	81/19
0.4	3,871	100%	123	53/47
0.5	3,811	100%	81	50/50
0.6	3,925	100%	101	39/61
0.8	4,043	100%	132	20/80
1.0	4,260	100%	212	0/100

Table 5 shows that increasing the fraction of human agents p generally increases total travel time and substantially increases the number of capacity-violation attempts. Even though all runs complete (completion stays at 100%), the rising violation-attempt count indicates that non-coordinating, occasionally-yielding humans more frequently attempt to enter saturated nodes/edges, forcing additional waiting and increasing system-wide travel time.

Notably, total travel time degrades overall from 3,484 at $p = 0$ (all AVs) to 4,260 at $p = 1$ (all humans), while violations rise from 0 to 212. This suggests that GLC remains effective in mixed-autonomy settings, but overall efficiency depends on the extent of compliance with local coordination rules.

A.3 Learning-Based Methods

We trained a policy on the 100-agent Manhattan instance by behavior cloning GLC trajectories. With a 10-minute runtime cutoff, only 7 out of 100 agents completed. This suggests that the learned policy did not reliably reproduce the local coordination behavior required at execution time. Training a policy from scratch without behavior cloning produced even worse results. Further study is needed to improve learning-based methods for this road-network coordination setting.

A.4 Additional MAPF Methods

We tested other recent MAPF baselines, including RHCR [24] and LaCAM [30], with a 10-minute cutoff on the same instance; RHCR completed 1 out of 100 agents and LaCAM completed 0 out of 100 agents. We interpret these outcomes primarily as evidence of problem mismatch: these methods are designed mainly for grid or free-space MAPF formulations, whereas our benchmark imposes directed road-network topology and capacity-constrained traffic dynamics.